

03-01 开发环境搭建手册

系统名称：智慧能源管理系统 系统版本：V4.6.0 文档版本：V1.0 编写日期：2026-09-14 适用对象：新入职后端 / 全栈研发人员 操作系统：Windows 10/11（命令以 PowerShell 为主）

目录

- 1. 硬件与软件清单
- 2. 中间件启动（Docker Compose）
- 3. 数据库初始化
- 4. 后端工程搭建与启动
- 5. 前端工程搭建与启动
- 6. MQTT 设备模拟器使用
- 7. Vue3 重构版 / uniapp（可选）
- 8. 端到端自测清单
- 9. 常见问题 FAQ（高频坑）
- 10. 开发工具推荐

1. 硬件与软件清单

1.1 硬件建议

项	最低配置	推荐配置
CPU	4 核	8 核及以上（本地同时跑 5 个中间件容器 + IDEA + Chrome）
内存	16 GB	32 GB (MySQL/Redis/RabbitMQ/EMQX/TDengine 全量容器较吃内存)
磁盘	30 GB 可用空间	50 GB 以上 SSD，Docker 镜像与数据卷建议放 SSD
网络	可访问外网（拉取	配置好内网代理或阿里云

项	最低配置	推荐配置
	Maven/npm/Docker 镜像)	镜像
1.2 必装软件		
软件	版本要求	说明
JDK	JDK 1.8 (强制)	根 pom.xml 中 java.version=1.8, 编译目标 1.8。 不兼容 JDK 17+ 直接启动 , 高版本必须加 --add-opens 参数 (见 4.6)。推荐 Temurin/Zulu JDK 8u372+
Maven	3.6.3 ~ 3.8.x	多模块构建; 根 pom 已开启并行构建 (4 线程)
Git	2.30+	克隆代码、分支管理
Node.js	推荐 16.x / 18.x LTS	前端为 Vue CLI 4 + Vue 2.6.12; 如使用 Node 20/22, dev 脚本已内置 - -openssl-legacy-provider, 不要自行删除
Docker Desktop	4.20+ (含 Compose V2)	本地运行 MySQL/Redis/RabbitMQ/EMQX/TDengine; Windows 需启用 WSL2
IntelliJ IDEA	2022.2+ (Ultimate 推荐)	安装 Lombok、Maven Helper 插件; Community 版可配合外置数据库工具
Python	3.8+ (仅模拟器需要)	运行 simulator/ MQTT 设备模拟与压测工具
数据库客户端	任选	DBeaver / Navicat / IDEA Database 工具
MQTT 客户端	任选	MQTTX、EMQX Dashboard 自带 WebSocket 工具

1.3 JDK 安装与 JAVA_HOME

1. 安装 JDK 8 后配置环境变量 (PowerShell 查看) :

```
java -version
# 期望输出: java version "1.8.0_xxx"
echo $env:JAVA_HOME
# 例: C:\Program Files\Eclipse Adoptium\jdk-8.0.392.8-hotspot
```

2. JAVA_HOME 指向 JDK 8 根目录; Path 中加入 %JAVA_HOME%\bin。
3. IDEA 中两处都要设置:
 - File → Project Structure → Project SDK = 1.8, Language level = 8;
 - Settings → Build, Execution, Deployment → Build Tools → Maven → Runner → JRE = 1.8。

1.4 Maven 安装与阿里云镜像

Maven settings.xml (%USERPROFILE%\.m2\settings.xml) 建议配置阿里云镜像, 避免依赖下载缓慢:

```
<settings>
  <mirrors>
    <mirror>
      <id>aliyun-public</id>
      <mirrorOf>*</mirrorOf>
      <name>aliyun public</name>
      <url>https://maven.aliyun.com/repository/public</url>
    </mirror>
  </mirrors>
  <profiles>
    <profile>
      <id>jdk8</id>
      <activation>
        <activeByDefault>true</activeByDefault>
        <jdk>1.8</jdk>
      </activation>
      <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
      </properties>
    </profile>
  </profiles>
</settings>
```

根 pom 已声明 maven.aliyun.com/repository/public 仓库，即使不配置 settings 也能走阿里云，但 settings 镜像对插件下载同样生效，建议配置。

1.5 Node.js 安装

推荐使用 nvm-windows 安装 Node 16/18:

```
nvm install 18.20.4
nvm use 18.20.4
node -v    # v18.x
npm -v
```

zhurong-admin-ui/package.json 的脚本已内置 set NODE_OPTIONS=--openssl-legacy-provider，因此在 Node 17+ 环境下也可直接 npm run dev。若直接调用 vue-cli-service 报错 error:0308010C:digital envelope routines::unsupported，手动执行：

```
$env:NODE_OPTIONS="--openssl-legacy-provider"
```

2. 中间件启动 (Docker Compose)

2.1 两种本地中间件形态 (先读!)

本项目本地研发有两种等价形态，二选一：

- **形态 A: shared-infra 共享中间件 (团队约定, 推荐)** 中间件统一收口在 D:\code\shared-infra 工程中启动 (容器名 shared-mysql、shared-redis、shared-rabbitmq、shared-emqx、shared-tdengine)，本系统通过 localhost 映射端口连接。后端配置 zhurong-ems-admin/src/main/resources/application-local.yml 即按此约定编写。
- **形态 B: 本仓库自带 compose** 在仓库根目录执行 docker compose up -d，使用根目录 docker-compose.yml 拉起 zhurong-ems-* 系列容器。

⚠ 账号差异注意：仓库自带 compose 中 RabbitMQ 默认账号是 guest/guest、Redis 未显式设置密码，而 application-local.yml 期望连接的是 **RabbitMQ rabbitmq/rabbitmqpassword、vhost admin_vhost、Redis 密码 difyai123456 (db 13)**。若使用形态 B，需自行在 RabbitMQ 中创建用户与 vhost、给 Redis 加密码，或临时修改 local 配置；团队统一使用形态 A 时无需任何调整。

2.2 启动命令

形态 A: 进入共享中间件目录 (具体以 *shared-infra* 工程为准)

```
cd D:\code\shared-infra
```

```
docker compose up -d
```

形态 B: 使用本仓库 *compose*

```
cd D:\code\gitcp\inspur-ems\deep-ems0
```

```
docker compose up -d
```

查看容器状态:

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

2.3 中间件服务说明 (镜像 / 端口 / 账号 / 健康检查 / 管理台)

以下镜像版本与根 *docker-compose.yml* 完全一致:

服务	镜像	主机端口	账号/口令	健康检查	管理台 / 备注
MySQL	mysql:8.0.35	3306	root / 123456, 初始化库 autoee_ems	mysqladmin ping -h localhost -uroot -p123456, 10s/次, 5次	字符集 utf8mb4, mysql_native_password, max_connections=1000
Redis	redis:7.2.3-alpine	6379	密码 difyai123456, 应用使用 db 13	redis-cli ping	命令行: docker exec -it shared-redis redis-cli -a difyai123456
RabbitMQ	rabbitmq:3.12-management-alpine	5672 (AMQP) 15672 (管理台)	应用连接: rabbitmq / rabbitmqpassword, vhost=admin_vhost (用户对该 vhost 拥	rabbitmq-diagnostics ping	http://localhost:15672

服务	镜像	主机端口	账号/口令	健康检查	管理平台 / 备注
EMQX (MQTT)	emqx/emqx: 5.3.2	1883 (MQTT) 8883 (MQTT/SSL) 18083 (Dashboard) 8083 (WS/SSL) 8081 (WS)	有 .* 权限) admin / public	emqx_ctl status	Dashboard : http://localhost:18083
TDengine (可选)	tdengine/tdengine:3.2.0.0	6030 (原生连接) 6041 (RESTful)	root 默认口令 taosdata; 库 energy	taos -c "show databases"	本地默认关闭, 由 --spring.datasource.dynamic.data-source.td.enabled=false 控制

2.4 应用期望的连接参数速查 (localhost)

组件	连接串	凭据
MySQL	jdbc:mysql://localhost:3306/autoee_ems	root / 123456
Redis	localhost:6379, database 13	密码 difyai123456
RabbitMQ	localhost:5672, virtual-host admin_vhost	rabbitmq / rabbitmqpassword
MQTT	tcp://localhost:1883	admin / public, clientId=zhurong-ems-local-client, QoS=1
TDengine (开启时)	jdbc:TAOS-RS://localhost:6041/energy	root / taosdata

逐组件探活:

```
# MySQL
docker exec -it shared-mysql mysql -uroot -p123456 -e "show databases;"
# Redis
docker exec -it shared-redis redis-cli -a difyai123456 -n 13 ping # PONG
# RabbitMQ
curl http://localhost:15672 # 管理平台可登录
# EMQX Dashboard
curl http://localhost:18083
```

RabbitMQ 需要的 vhost/用户若尚未创建，在管理平台 Admin 页创建 vhost admin_vhost，新建用户 rabbitmq/rabbitmqpassword 并授予该 vhost 的 .* 权限。

3. 数据库初始化

3.1 全量业务库（必做）

全量结构与基础数据脚本：.dev/dumps/ems_full.sql。该 dump 由 mysqldump 导出，已包含 **CREATE DATABASE IF NOT EXISTS autoee_ems** 与 **USE autoee_ems**，无需手工预建库。

方式一：命令行导入

```
# 在仓库根目录
Get-Content .dev\dumps\ems_full.sql -Raw | docker exec -i shared-mysql
mysql -uroot -p123456
# 或先拷贝进容器再导入
docker cp .dev/dumps/ems_full.sql shared-mysql:/tmp/ems_full.sql
docker exec shared-mysql sh -c "mysql -uroot -p123456 -h127.0.0.1 < /tmp/ems_full.sql"
```

方式二：DBeaver/Navicat 连接 localhost:3306 (root/123456)，选中数据库连接执行 ems_full.sql。

3.2 医院智慧能源模块脚本

医院模块为**加性开发**（只新增表，不改动原有表），脚本位于仓库 数据库脚本/ 目录：

脚本	作用	目标库
hospital_init.sql	医院 9 张核心表： hospital_device、 hospital_metric_def、	MySQL autoee_ems

脚本	作用	目标库
	hospital_device_data、 hospital_area、 hospital_alarm_rule、 hospital_alarm_record 、 hospital_callback_log 、 hospital_device_workload 等	
hospital_menu.sql、 hospital_menu_m2.sql、 hospital_menu_m3.sql、 hospital_menu_m4.sql	医院模块菜单与按钮权限标识（按迭代追加，需全部执行）	MySQL autoee_ems
hospital_alarm.sql	报警相关补充表/数据	MySQL autoee_ems
hospital_seed_demo.sql	演示设备/指标/规则种子数据（本地联调推荐导入）	MySQL autoee_ems
hospital_role_dashboard.sql	医院角色与大屏看板权限	MySQL autoee_ems
hospital_init_tengine.sql	TDengine 超表 hospital_device_data （仅开启 TD 时需要）	TDengine 库 energy
hospital_rename_reserved_words.sql	历史保留字段修正脚本（存量库升级用）	MySQL autoee_ems
# 示例：逐个导入医院 MySQL 脚本		
docker cp 数据库脚本/hospital_init.sql shared-mysql:/tmp/		
docker exec shared-mysql sh -c "mysql -uroot -p123456 autoee_ems < /tmp/hospital_init.sql"		

其他常用脚本：

- 根库初始化：数据库脚本/init.sql；
- XXL-Job 调度中心表：docker/mysql/init_xxl_job_tables.sql、docker/mysql/tables_xxl_job.sql，目标库 xxl_job（仅联调定时任务调度中心时需要）；
- fix_charging_pile.sql、fix_sys_menu_encoding.sql 等为历史数据修复脚本，全新库不需要执行。

3.3 TDengine 初始化（默认跳过）

本地默认不启用 TDengine，本节可跳过。需要时序库时：

1. 启动参数去掉 --

```
spring.datasource.dynamic.datasource.td.enabled=false（或置 true）；
```

2. 在 energy 库执行 数据库脚本/hospital_init_tdengine.sql，创建超表：

```
CREATE STABLE IF NOT EXISTS hospital_device_data (  
    ts TIMESTAMP, metric_value DOUBLE, quality INT  
) TAGS (device_code NCHAR(64), metric_code NCHAR(64));
```

TDengine 3.3.x 注意：强制密码复杂度（需大小写+数字/特殊字符）、不支持 CREATE USER IF NOT EXISTS；不要用 TAOS_ROOT_PASSWORD 覆盖 root 口令、不要自定义 command 覆盖镜像自带 entryptpoint，否则会初始化崩溃循环重启。详见 EMS 接入共享中间件本地研发联调经验.md。

3.4 导入校验

执行 .dev/dumps/verify.sql:

```
SELECT table_schema, COUNT(*) AS tbl_cnt  
FROM information_schema.tables  
WHERE table_schema IN ('autoee_ems','xxl_job')  
GROUP BY table_schema;
```

并抽查关键表：

```
USE autoee_ems;  
SHOW TABLES LIKE 'hospital%';           -- 医院表已建  
SELECT COUNT(*) FROM sys_user;            -- 含 admin 用户  
SELECT COUNT(*) FROM sys_menu;            -- 菜单已导入
```

4. 后端工程搭建与启动

4.1 获取代码

```
cd D:\code\gitcp\inspur-ems  
git clone <仓库地址> deep-ems0  
cd deep-ems0
```

4.2 Maven 模块说明

根 pom.xml 定义 groupId=com.zhurong-ems、version=4.6.0，Spring Boot 2.7.9，打包方式 pom，当前参与构建的模块：

模块	职责
zhurong-ems-admin	启动模块 + Controller 层。启动类 cpEmsApplication.java (com.cpems 包)，所有 Web 控制器在 com.cpems.web.controller.*
zhurong-ems-framework	框架核心：Sa-Token 安全配置、拦截器、AOP、全局异常处理、跨域、XSS 过滤等
zhurong-ems-system	业务主体 ：domain / service / mapper，包名 com.ruoyi.system.*，含 com.ruoyi.system.hospital 医院子域、MQTT/MQ 链路 config
zhurong-ems-job	XXL-Job 执行器： com.cpems.job.config.XxlJobConfig，受 xxl.job.enabled 开关控制
zhurong-ems-common	公共层：R、TableDataInfo、BaseController、工具类、注解、异常、实体基类
zhurong-ems-extend	扩展服务（独立启动）：zhurong-ems-xxl-job-admin（调度中心，启动类 XxlJobAdminApplication）、zhurong-ems-monitor-admin（Spring Boot Admin 监控）
zhurong-ems-oss	对象存储（S3 协议，aws-java-sdk-s3）
zhurong-ems-sms	短信（阿里云 dysmsapi / 腾讯云 sms SDK）
zhurong-ems-generator	代码生成器（Velocity 模板）， 根 pom 中默认注释未启用
zhurong-ems-demo	演示模块，默认不参与构建

关键依赖版本：MyBatis-Plus 3.5.3.1、dynamic-datasource 3.5.2、Sa-Token 1.34.0、Hutool 5.8.15、Redisson 3.20.0、Lock4j 2.2.3、XXL-Job 2.3.1、EasyExcel 3.2.1、fastjson2 2.0.53、Lombok 1.18.36。

4.3 命令行构建

在仓库根目录：

```
# 首次构建 (跳过测试, 最快)
mvn clean package -DskipTests
```

```
# 按 local profile 构建 (资源文件使用 local 变量过滤)
mvn clean package -Plocal -DskipTests
```

构建成功后产物: zhurong-ems-admin/target/zhurong-ems-admin.jar。

辅助脚本 (Linux/容器构建用, Windows 本地了解即可) :

- build_backend.sh: 容器内安装 Maven 并查看版本;
- build_full.sh: 容器内执行 mvn clean package -DskipTests -B。

4.4 配置文件说明

后端配置位于 zhurong-ems-admin/src/main/resources/:

文件	用途
application.yml	公共配置: 端口、Sa-Token、MyBatis-Plus、Swagger、医院 IOT 回调参数、XSS、线程池等
application-local.yml	本地研发自包含配置: 指向 localhost 的 shared-infra 中间件, 含 HikariCP/Redis(db13)/RabbitMQ/MQTT/Redisson, 关闭 xxl-job 与 SBA 注册
application-dev.yml	开发服务器配置 (默认 profile, 含 td 数据源模板, 默认 td.enabled=false)
application-prod.yml / application-docker.yml / application-test.yml	生产 / Docker / 测试环境

仓库根目录还放有一份同名 application-local.yml 覆盖样例, **真正生效的是 admin 模块 resources 下的文件**; 根目录样例供容器挂载或排障对照。

application-local.yml 关键项 (已按 shared-infra 约定配好, 一般无需修改) :

- MySQL: localhost:3306/autoee_ems, root/123456, Hikari 连接池;
- Redis: localhost:6379, **database 13**, 密码 difyai123456; Redisson 单节点配置同步带密码;
- RabbitMQ: localhost:5672, rabbitmq/rabbitmqpassword, vhost admin_vhost;

- MQTT: tcp://localhost:1883, admin/public, clientId=zurong-ems-local-client, QoS=1, keepalive=360;
- xxl.job.enabled=false、spring.boot.admin.client.enabled=false (本地无调度中心/监控中心, 避免启动即连不上)。

公共 application.yml 中医院回调参数:

```
hospital:
  iot:
    auth-token: hospital-iot-2026          # 回调请求头 X-IOT-Token
    ip-whitelist:                          # 为空不限制
    sign-enabled: false                    # HMAC-SHA256 签名开关
    sign-secret:
    monitor-offline-minutes: 30
    exchange: hospital.topic.exchange
    queue: hospital.device.data.queue
    routing-key: hospital.device.data
```

4.5 IDEA 启动配置 (推荐方式)

1. 用 IDEA 打开仓库根目录 (识别根 pom 后自动导入全部 Maven 模块) ;
2. 找到启动类 cpEmsApplication.java (包 com.cpems, 注解 @SpringBootApplication @EnableAsync @EnableScheduling @ComponentScan(basePackages={"com.cpems","com.ruoyi"})) ;
3. 新建 Application Run Configuration:
 - **Main class:** com.cpems.cpEmsApplication;
 - **Active profiles:** local;
 - **Program arguments:**

```
--spring.profiles.active=local --server.port=8088 --server.servlet.context-path=/autoee-iot-ems --spring.datasource.dynamic.datasource.td.enabled=false
```

- JDK 选 1.8;
- 4. 启动后控制台出现启动横幅, 打印 Local:

http://localhost:8088/autoee-iot-ems, 并出现 MQTT 连接成功, 连接方式: 直连 日志。

4.6 高版本 JDK 启动 (-add-opens)

如本机只有 JDK 9+ (11/17/21) 并尝试启动, CGLIB 反射会抛 InaccessibleObjectException, 表现为全站接口 500。必须在 VM options 中追加以下 **10 项** --add-opens:

```
--add-opens java.base/java.lang=ALL-UNNAMED
--add-opens java.base/java.lang.reflect=ALL-UNNAMED
--add-opens java.base/java.math=ALL-UNNAMED
--add-opens java.base/java.util=ALL-UNNAMED
--add-opens java.base/java.util.concurrent=ALL-UNNAMED
--add-opens java.base/java.net=ALL-UNNAMED
--add-opens java.base/java.io=ALL-UNNAMED
--add-opens java.base/java.nio=ALL-UNNAMED
--add-opens java.base/sun.nio.ch=ALL-UNNAMED
--add-opens java.base/sun.security.ssl=ALL-UNNAMED
```

命令行整行启动（高版本 JDK 场景，与 AGENTS.md 一致）：

```
java --add-opens java.base/java.lang=ALL-UNNAMED `
  --add-opens java.base/java.lang.reflect=ALL-UNNAMED `
  --add-opens java.base/java.math=ALL-UNNAMED `
  --add-opens java.base/java.util=ALL-UNNAMED `
  --add-opens java.base/java.util.concurrent=ALL-UNNAMED `
  --add-opens java.base/java.net=ALL-UNNAMED `
  --add-opens java.base/java.io=ALL-UNNAMED `
  --add-opens java.base/java.nio=ALL-UNNAMED `
  --add-opens java.base/sun.nio.ch=ALL-UNNAMED `
  --add-opens java.base/sun.security.ssl=ALL-UNNAMED `
  -jar zhurong-ems-admin/target/zhurong-ems-admin.jar `
  --spring.profiles.active=local `
  --server.port=8088 `
  --server.servlet.context-path=/autoee-iot-ems `
  --spring.datasource.dynamic.datasource.td.enabled=false
```

团队仍建议正式编译/运行使用 JDK 8，避免其他隐性兼容问题。

4.7 脚本启动

根目录 start-backend.bat 为快速启动脚本（注意其端口为 8080、未设置 context-path，适合极简联调；前端默认对接的是 8088 + /autoee-iot-ems，日常开发请优先使用 4.5 的 IDEA 配置）：

```
java -jar zhurong-ems-admin\target\zhurong-ems-admin.jar --spring.profi
les.active=local --server.port=8080 --spring.datasource.dynamic.datasou
rce.td.enabled=false
```

4.8 启动验证

```
# 健康检查 (actuator 已放开, 且 show-details=ALWAYS)
curl http://localhost:8088/autoee-iot-ems/actuator/health
# 期望: {"status":"UP",...}, components 中 db/redis/rabbit 均 UP

# 验证码接口 (无需登录)
curl "http://localhost:8088/autoee-iot-ems/captchaImage"
```

日志关键标志：

- 服务启动成功！当前环境：local，打印 Local/External/dbUrl；
- MQTT 连接成功，连接方式：直连（MyMQTTCallback.java）；
- RabbitMQ 监听器容器启动，监听 electric/voltage、electric/current、electric/power、electric/consumption、water/consumption/+ 队列；
- 无 xxl-job 注册失败报错（local 已关闭）。

接口文档（springdoc）：启动后访问 Swagger UI（分组含「2.系统模块 com.cpems.web」），地址见控制台输出 /swagger-ui/index.html。

5. 前端工程搭建与启动

5.1 安装依赖并启动

```
cd zhurong-admin-ui
npm install
npm run dev
```

- 访问地址：**http://localhost:9029**（端口在 vue.config.js 中固定为 9029）；
- 技术栈：Vue 2.6.12 + Vue CLI 4.4.6 + Element UI 2.15.12 + Vuex 3 + vue-router 3 + axios 0.24 + ECharts 5.4 + vue-i18n 8 + sass 1.32；
- dev 脚本内容为 set NODE_OPTIONS=--openssl-legacy-provider && vue-cli-service serve，高版本 Node 可直接用。

5.2 后端地址配置

.env.development:

```
VUE_APP_TITLE = 智慧能源监控平台
VUE_APP_BASE_API = 'http://localhost:8088/autoee-iot-ems'
```

src/utils/request.js 以该变量为 axios baseURL，并自动加请求头 Authorization: Bearer <token>、Content-Language: zh_CN。后端端口/context-path 若变更，需同步修改此文件，不要在业务代码中硬编码后端地址。

5.3 生产构建与 Nginx 部署

```
npm run build:prod      # 产物输出到 dist/
npm run build:test      # test 模式构建
npm run lint            # ESLint 检查 src 下 .js/.vue
```

容器化部署文件：

- Dockerfile: Node 构建 + Nginx 运行;
- nginx.conf: 监听 80, 静态资源根目录 /usr/share/nginx/html, /prod-api/、/dev-api/、/autoee-iot-ems/ 均反代到 http://zhurong-ems-backend:8088/autoee-iot-ems/。

5.4 Vue2 模板铁律

属性位置必须使用 **v-bind**, 禁止在 **HTML** 属性内写 **mustache**:

```
<!-- 正确 -->
<chart :yName="$t('hospital.power')" />
<!-- 错误: 编译直接失败, 且会阻断整个应用编译 -->
<chart yName="{{ $t('hospital.power') }}" />
```

全量排查命令 (PowerShell 可用 ripgrep) :

```
rg '"{{' src --glob "*.vue"
```

6. MQTT 设备模拟器使用

模拟器位于 simulator/, Python 实现, 可向 EMQX 发送单条报文或执行压测。

6.1 安装与配置

```
cd simulator
pip install -r requirements.txt    # 核心依赖 paho-mqtt、PyYAML
```

编辑 config/config.yaml, 将 host 改为本地 EMQX:

```
mqtt:
  host: "tcp://localhost:1883"
  username: "admin"
  password: "public"
  client_id: "mqtt-simulator-${HOSTNAME}"
  qos: 0
stress_test:
  scenarios:
    light: {devices: 10, frequency: 1, duration: 60, message_type:
"voltage"}
    medium: {devices: 100, frequency: 5, duration: 300, message_type:
"voltage"}
    heavy: {devices: 1000, frequency: 10, duration: 600, message_type:
"mixed"}
```

6.2 常用命令

Windows 直接双击 run.bat（自动检测 Python 与依赖），或命令行：

```
python main.py ping           # 测试与 EMQX 的连通性
python main.py topics         # 查看支持的主题
python main.py templates      # 查看消息模板 (templates.yaml)
python main.py validate       # 校验配置

# 发送单条电压数据 (主题需与后端订阅一致)
python main.py send -t electric/voltage -cid device_001 -tpl voltage
# 打包数据 (数组) 走 electric/all/{clientId}
python main.py send -t electric/all/device_001 -tpl electric_all

# 压力测试: light/medium/heavy 三档场景
python main.py stress -s light
```

后端订阅的主题 (TopicType.java)：electric/emsCarson、electric/all/+、electric/voltage、electric/current、electric/power、electric/consumption、water/consumption/+。

发送后在后端日志可见 接收消息主题：electric/voltage... → 已转发数据到... 的完整链路日志；压测报告输出到 simulator/output/，运行日志在 simulator/logs/。

7. Vue3 重构版 / uniapp（可选）

7.1 Vue3 重构版 zhurong-admin-ui-v3

zhurong-admin-ui-v3/ 为现代化重构试点（Vue 3.4 + Vite 5 + Ant Design Vue 4 + TypeScript + Pinia + UnoCSS，包管理器 pnpm 9+）。当前主用前端仍是 **zhurong-admin-ui (Vue2)**，重构版按需启动：

```
cd zhurong-admin-ui-v3
pnpm install
pnpm dev           # 本地开发
pnpm build:prod    # 生产构建
pnpm lint          # ESLint
```

7.2 uniapp 移动端

zhurong-uniapp/ 为移动端工程（巡查、告警、设备等页面），使用 HBuilderX 打开目录运行到微信小程序 / App / H5；也可使用 uni-app CLI。接口配置见 zhurong-uniapp/config.js 与 utils/request.js。

8. 端到端自测清单

按顺序执行，全部通过即代表本地环境完整可用。

8.1 中间件与数据库

- ☐ docker ps 中 MySQL/Redis/RabbitMQ/EMQX 均 healthy;
- ☐ verify.sql 能查到 autoee_ems 表数量；医院表 SHOW TABLES LIKE 'hospital%'; 存在。

8.2 后端启动

- ☐ IDEA 以 local profile 启动成功，无堆栈；
- ☐ curl http://localhost:8088/autoee-iot-ems/actuator/health 返回 UP;
- ☐ 日志出现 MQTT 连接成功，连接方式：直连。

8.3 前端登录

- ☐ http://localhost:9029 打开登录页；
- ☐ 使用 **admin / 123456** 登录（数学验证码按页面提示输入）；
- ☐ 登录后菜单正常加载（动态菜单由 /getRouters 下发），无 401 循环、无白屏。

8.4 MQTT 直采链路（设备 → EMQX → RabbitMQ → MySQL/报警）

- ☐ 模拟器发送：python main.py send -t electric/voltage -cid device_001 -tpl voltage;
- ☐ 后端日志依次出现：接收消息主题 → 转发 EquipmentTopicExchange → [开始]-接收到 RabbitMQ 的消息-进行业务处理；
- ☐ 对应业务表（实时数据/设备数据）出现新记录；配置了报警规则且越限时，实时报警表生成记录并触发邮件/工单逻辑；
- ☐ EMQX Dashboard (http://localhost:18083) 可看到客户端连接与主题流量。

8.5 医院 IOT 回调链路 (curl 自测)

先在「医院设备台账」中把 IOT 平台设备 ID (如 `iot-device-001`) 绑定到本系统设备, 再执行 (PowerShell, 注意 Windows PowerShell 5.1 对 JSON body 的引号处理, 建议使用 Postman 或 PowerShell 7) :

```
curl -X POST "http://localhost:8088/autoee-iot-ems/hospital/callback/data" `
-H "Content-Type: application/json" `
-H "X-IOT-Token: hospital-iot-2026" `
-d '{"msgId":"msg-001","timestamp":"2026-09-14 10:00:00","devices":[{"deviceId":"iot-device-001","points":[{"metric":"power","value":12.34,"ts":"2026-09-14 10:00:00","quality":0}]}]}'
```

- ☐ 返回 `{"code":200,...,"data":"OK"}`;
- ☐ `hospital_callback_log` 表新增一条成功日志 (含 `request_id`、`point_count`、`cost_time`) ;
- ☐ `hospital_device_data` 表出现数据点;
- ☐ 命中阈值规则时 `hospital_alarm_record` 产生报警;
- ☐ Token 错误时返回失败, 日志状态为鉴权失败 (`/hospital/callback/**` 在 `Sa-Token` 排除路径中, 无需登录, 但 `X-IOT-Token` 必校验) 。

8.6 前端业务页面

- ☐ 实时监控、能耗分析、医院监测页 (设备在线/离线, 离线阈值 30 分钟) 可正常打开;
- ☐ 按钮权限正常 (无权限标识的按钮不渲染, 指令 `v-hasPermi`) 。

9. 常见问题 FAQ (高频坑)

9.1 所有页面/接口报 500: InaccessibleObjectException

原因: 使用 JDK 9+ 启动, 模块封装禁止 CGLIB 反射访问 `java.lang.ClassLoader` 等。 **解决:** 换 JDK 8; 或按 4.6 加齐 10 项 `--add-opens`。详见 打包部署常见问题.md。

9.2 特定模块 500: 表不存在

现象: 日志 `Table 'autoee_ems.xxx' doesn't exist`。 **解决:** 从 `.dev/dumps/ems_full.sql` 补对应建表 DDL; 医院表缺失去 数据库脚本/ 执行 `hospital_init.sql` 及对应菜单脚本。

9.3 逻辑删除与主键

- 逻辑删除字段固定 `del_flag`: 0=存在, 2=删除; 实体字段使用 `@TableLogic`, 全局配置 `logicDeleteValue=2 / logicNotDeleteValue=0`, 查询不要手工拼 `del_flag`;
- 主键统一雪花算法: 全局 `id-type: ASSIGN_ID`, 实体 `@TableId(value="xxx_id")`, 不要在 insert 时手工填主键 (子表初始化脚本中 id 需显式给出长整型值)。

9.4 Redis 连接报错 / NOAUTH / db 不对

原因: 共享 Redis 必须带密码 `difyai123456`, 应用使用 db 13。解决: 检查 `application-local.yml` 的 `spring.redis.password` 与 `redisson.singleServerConfig.password` 是否一致; 命令行连接加 `-a difyai123456 -n 13`。

9.5 RabbitMQ 连接 / 通道启动失败

- 确认 `vhost admin_vhost` 已创建, 用户 `rabbitmq` 对其有配置/写/读权限;
- 队列由应用启动时自动声明 (`RabbitExchangeConfig`、`HospitalRabbitConfig`), 但 `vhost`/用户必须先存在;
- 本地不用消息链路时也不要吧 RabbitMQ 停掉, 否则监听器容器会持续重连刷错误日志。

9.6 TDengine 初始化 / 启动崩溃

TDengine 3.3.x: 密码需满足复杂度、不支持 `CREATE USER IF NOT EXISTS`; 不要用 `TAOS_ROOT_PASSWORD` 覆盖 `root`、不要自定义 `command` 覆盖镜像 `entrypoint`。本地默认 `td.enabled=false`, 不启用即可规避; 排障见 TDengine 手动初始化操作手册.md 与共享中间件联调经验文档。

9.7 其他高频小问题

现象	处理
前端启动报 <code>digital envelope routines::unsupported</code>	使用 Node 16/18, 或保留脚本内 <code>--openssl-legacy-provider</code>
前端编译卡在某个 <code>.vue</code> 文件	多为属性内 <code>mustache</code> (<code>"{{ }}"</code>), <code>rg '"{{' src -g '*.vue'</code> 排查
<code>npm install</code> 慢	切淘宝 registry: <code>npm config set registry https://registry.npmirror.com</code>

现象	处理
8088/9029/3306 端口占用	<code>netstat -ano findstr :8088</code> 找进程，或调整启动端口
MQTT 一直连不上	确认 EMQX 容器在跑、Dashboard 能登录；clientId 不要与其他实例重复
登录后菜单空白	sys_menu 数据未导入全，或当前角色未分配菜单；检查 <code>/getRouters</code> 返回
xxl-job 注册失败刷错误日志	local 已 <code>xxl.job.enabled=false</code> ；若自定义配置确认不要误开
Maven 编译 Lombok/getter 找不到	IDEA 安装 Lombok 插件并开启 Annotation Processing；JDK 选 1.8

10. 开发工具推荐

类别	工具
IDE	IntelliJ IDEA Ultimate（后端 + 前端）、VS Code（前端/Python 辅助）
IDEA 插件	Lombok、Maven Helper、MyBatisX、Rainbow Brackets、Alibaba Java Coding Guidelines、Translation
数据库	DBeaver（免费）、Navicat、IDEA Database 面板
Redis	Another Redis Desktop Manager、RedisInsight
RabbitMQ	自带管理平台 http://localhost:15672
MQTT	MQTTX、EMQX Dashboard (http://localhost:18083)
接口调试	Apifox / Postman（导入 Swagger： <code>/v3/api-docs</code> ）
前端	Vue Devtools、FeHelper/Apifox 插件
Git	Git for Windows + Beyond Compare / IDEA 自带 Diff
容器	Docker Desktop + lazydocker / Portainer（可选）

类别

笔记/抓包

工具

Draw.io（架构图）、Wireshark/MQTTX
日志（协议排查）

搭建完成后，请继续阅读 03-02 编码规范与开发约定.md 与 03-03 模块
详细设计与实现说明.md。